UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Computación Grafica e

Iteración Humano Computadora

Final Project – Technical Manual

TEACHER:

Carlos Aldair Roman Balbuena

ACCOUNT NUMBER:

422113340

GROUP: 5

SEMESTER: 2026-1

DATE: 15 / 11 / 2025

Index

1. Objectives

The following objectives have been established:

1. Develop an interactive virtual tour using OpenGL that includes the realistic representation of a façade and two interior rooms, based on carefully selected reference images.
2. Integrate a synthetic camera that can be manipulated in real time, allowing free and natural exploration of the virtual space and providing an immersive experience for the user.

2. Project Scope

The scope of this project consists of designing and recreating an architectural façade using 3D modeling and animation techniques. For this purpose, a fictional space was selected: Marceline's house from the series *Adventure Time*. The choice of this structure allows for creative freedom.

The project includes the implementation of two rooms and a portion of the yard, focusing mainly on the façade as the central element. The scene will be built through a combination of:

- Models created directly in OpenGL using geometric primitives.
- Models created in Blender, exported, and integrated into the scene.

Additionally, five animations will be developed within the environment: three considered simple, based on basic geometric transformations (translation, rotation, and scaling), and two complex animations that will include more advanced movements, non-linear trajectories, or interactions between elements. The final result will be a dynamic, organized, and functional scene that adequately represents the chosen space and demonstrates proficiency in modeling, texturing, and animation within a 3D graphical environment programmed in OpenGL.

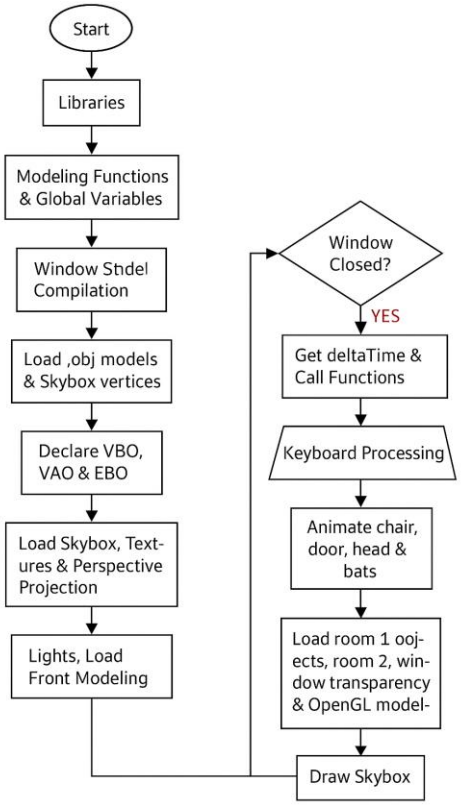
3. Limitations

To narrow the development and ensure the feasibility of the project within the given timeframe, the following limitations were established:

1. The modeling is restricted to a maximum of two rooms, avoiding expansion of the scene beyond what is necessary to maintain focus on the façade.
2. The recreated space will be limited to the façade and a portion of the yard, without including complex interiors or large additional structures.
3. All animations must be programmed directly in OpenGL, without using external animation systems or automated tools.
4. At least seven objects must be modeled using geometric primitives in OpenGL.
5. At least seven objects previously modeled in Blender must be integrated, respecting the export/import workflow to OpenGL.

6. Complex animations may not rely solely on simple geometric transformations or linear trajectories; they must include more elaborate behaviors, such as curves, combinations of movements, or interactions between objects.
7. Simple animations may be based on basic geometric transformations and linear trajectories, as their purpose is to demonstrate fundamental mastery of the animation pipeline in OpenGL.

4. Software flowchart



5. Gantt Chart

	Days	week 1	week 2	week 3	week 4	week 5	week 6	week 7	week 8
Project Approval	1								
Object Approval	1								
Modeling of 3 Objects	3								
Modeling of 4 Objects	3								
Advance on Facade	7								
Modeling of 3 Objects	14								
Room Modeling	14								
Texturing of 4 Objects	14								
Implementation in OpenGL	14								
Facade Texturing	14								
Texturing of 6 Objects	14								
Skybox Implementation	7								
Code Error Fixing	7								
Room Texturing	7								
Texture Corrections	7								
User Manual	7								
Technical Manual	7								
Cost Analysis	7								
Complete									
Incomplete									

6. Applied Software Methodology

At the beginning of the project, a traditional waterfall methodology was chosen, since its linear structure seemed appropriate for a clearly defined process:

1. Requirements gathering
2. Modeling design
3. Texturing
4. Implementation in OpenGL
5. Testing and corrections
6. Documentation

However, as the project progressed, an important limitation was identified: The waterfall model is rigid and does not adapt well to changes once a phase is formally closed.

During development, several adjustments became necessary, such as updates to 3D models, texture changes, compatibility issues with OpenGL, room reorganizations, and constant revisions of materials. This caused the sequential approach to slow down progress and forced the reopening of activities that should have already been closed according to the waterfall model.

Due to this issue, it became necessary to adopt a more flexible approach.

Scrum was selected as the agile framework, since it allows:

- Dividing the project into short sprints
- Continuously reviewing progress
- Delivering functional product increments
- Adjusting priorities without affecting the entire plan
- Integrating constant feedback from the professor

Scrum was especially helpful for tasks such as:

- Adjustments to 3D models
- Fixing textures that did not work correctly
- Animations that required iterations
- Testing in real execution within the OpenGL engine
- Progressive integration of lights, camera, skybox, and materials

This allowed the project to remain fully functional while new features continued to be added.

The project ultimately adopted a hybrid methodology:

1. Waterfall was used for the general and mandatory phases established from the start (approval, base modeling, texturing).
2. Scrum was applied within each major phase through sprints, to iterate improvements, fix errors, and adjust the product without disrupting the workflow.

This hybrid approach made it possible to maintain the formal structure of the waterfall model while benefiting from the flexibility of agile development.

The Gantt chart represents the project's initial sequential planning. Despite being based on waterfall, each multi-week block was internally divided into 7- or 14-day sprints, enabling iterations, reviews, and continuous adjustments.

7. Technologies Used and Project Interaction

For the development of the project, various software tools were necessary. Together, they enabled modeling, texturing, programming, compiling, and real-time visualization of the entire 3D environment. Below is a description of the technologies used and how they interact with each other throughout the complete project workflow.

7.1 3D Modeling in Blender

To create all the objects and structures in the project, Blender was chosen mainly for two reasons:

1. It is free software, which makes it easy to work without licensing restrictions.
2. It offers great freedom and control when modeling geometry, allowing each part of the scene to be created with precision.

However, as mentioned in class, Blender has a clear disadvantage in texture handling, since its UV Mapping system is not as intuitive as in other professional tools such as Maya, where texturing is a more comfortable and visually simple process. Throughout the project, several difficulties arose related to texture stretching and proper material assignment—issues that were not as common with Maya during laboratory practices. I cannot speak from direct experience regarding 3ds Max, since it was not used during this process.

Despite these challenges, Blender's efficiency in modeling and its economic accessibility made it the best option for the project.

7.2 Project Integration: Visual Studio + OpenGL

Once the models were completed, they were exported and integrated into the project through Visual Studio, the main development environment. This software was essential for:

- Writing and organizing the project's C++ code
- Managing resources, external libraries, and texture paths
- Compiling and running the program with direct support for graphics APIs

Due to the environment configuration, it was necessary to work on Windows, as Visual Studio provides more direct compatibility with the libraries used. I am not entirely sure if the entire configuration could be easily replicated in Linux, although it might require a different toolchain.

The project uses the OpenGL API along with libraries such as GLFW, GLAD, SOIL/STB_Image, and ASSIMP to handle window creation, model loading, shader initialization, and texture processing. Each library is not documented in detail here because all of them have extensive and sufficient official documentation available online.

7.3 Texture Editing and Generation

Another fundamental component of the project was the creation and editing of images to be used as textures on the 3D models. Different tools were used depending on the need:

- **From the phone:** PicsArt and Sketchbook
- **From the computer:** Microsoft Paint
- **AI tools:** ChatGPT (custom texture generation) and Gemini (gathering and cleaning reference images)

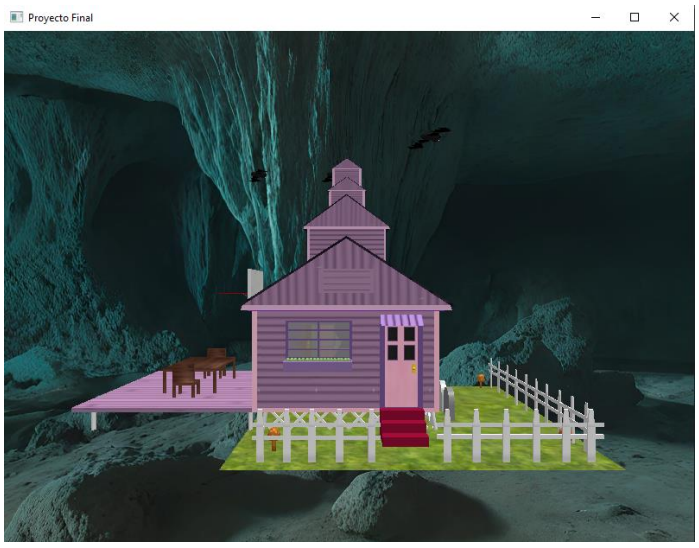
The combined use of traditional editors and artificial intelligence made it possible to obtain more detailed textures, correct visual errors, and generate images very similar to the original references of the project.

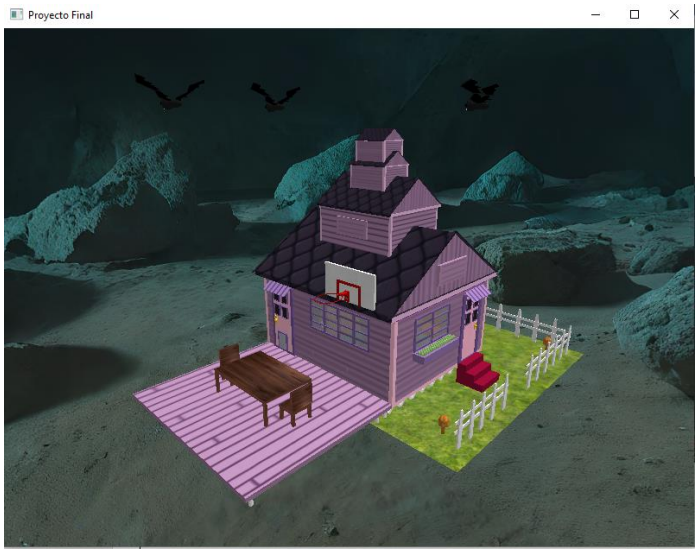
7.4 General Project Interaction

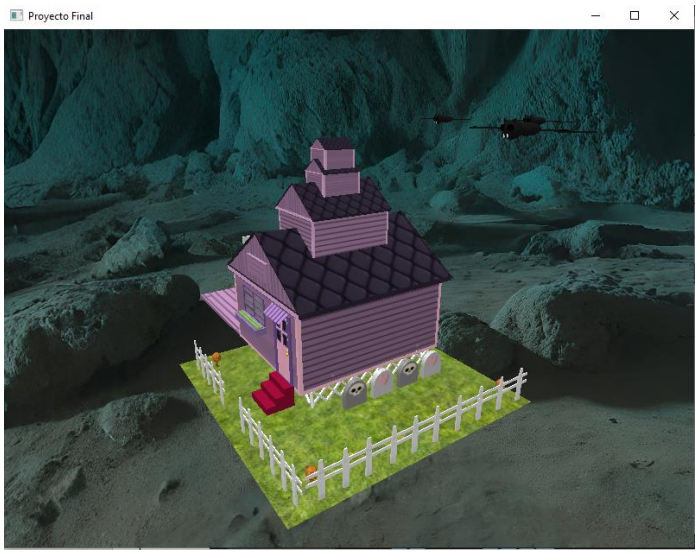
The interaction between all the technologies can be summarized in the following points:

1. 3D modeling in Blender
2. Exporting the model (OBJ/FBX/GLTF)
3. Editing or creating textures in PicsArt, Sketchbook, Paint, or AI tools
4. Importing models and textures into Visual Studio
5. Implementing C++ code for animations, lighting, camera, physics, and project logic
6. Processing through OpenGL, shaders, and transformation matrices
7. Final execution of the interactive 3D environment

8. Reference and modeling images


Proyecto Final

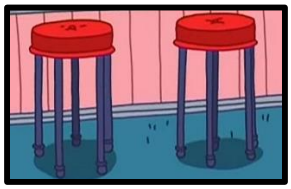

Proyecto Final

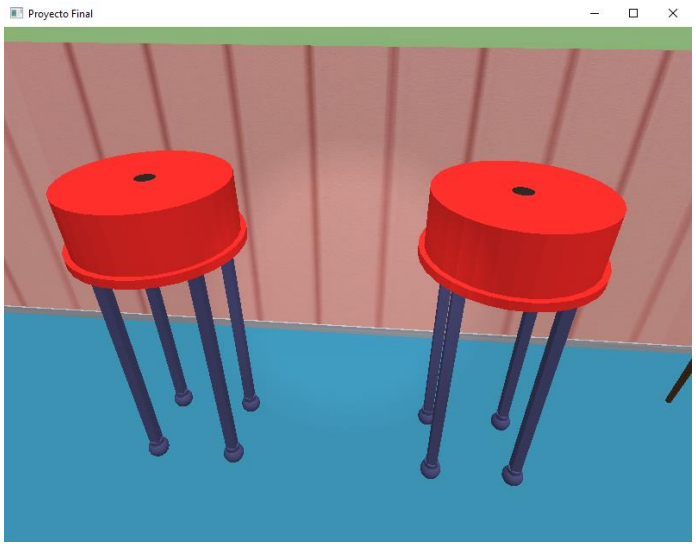

Proyecto Final

Room 1

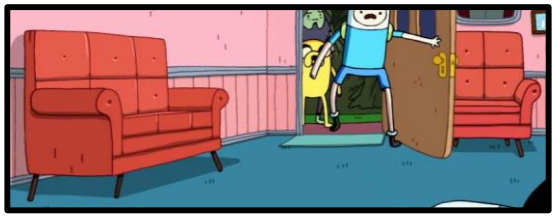
List of items in room 1:

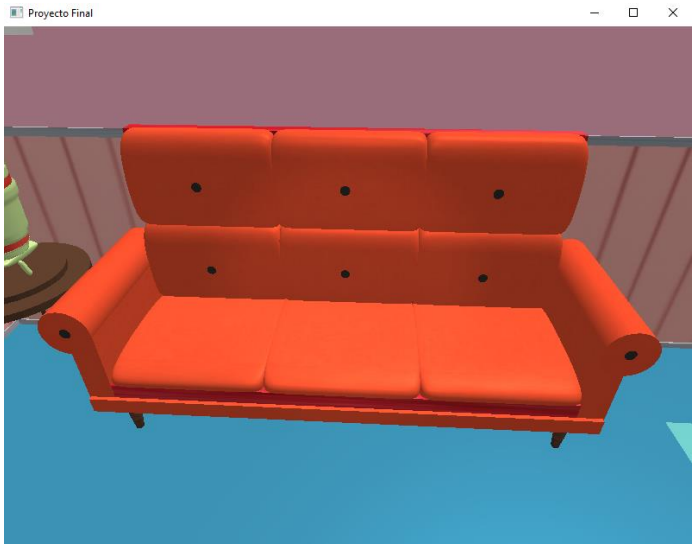
1. Object:





2. Object:



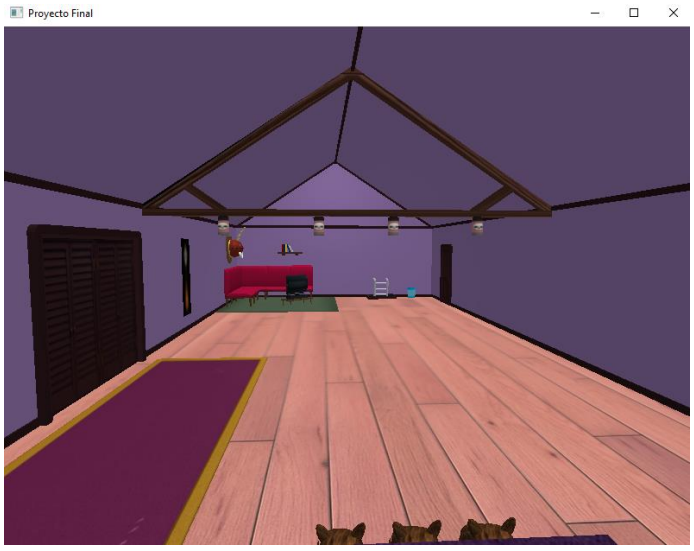


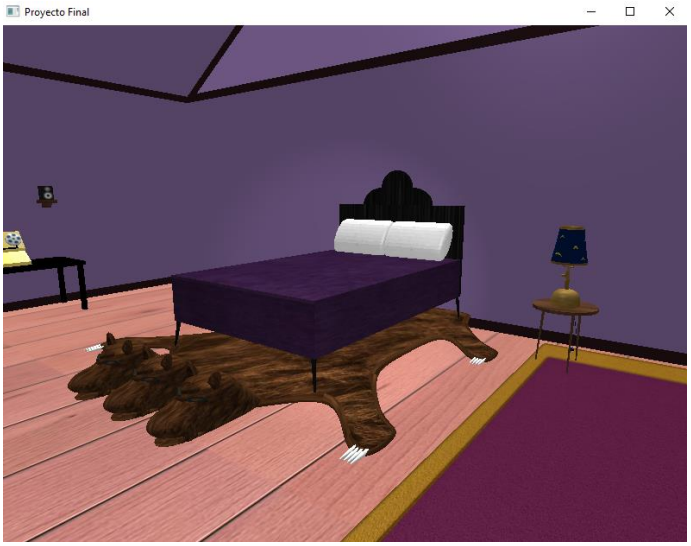
3. Object:

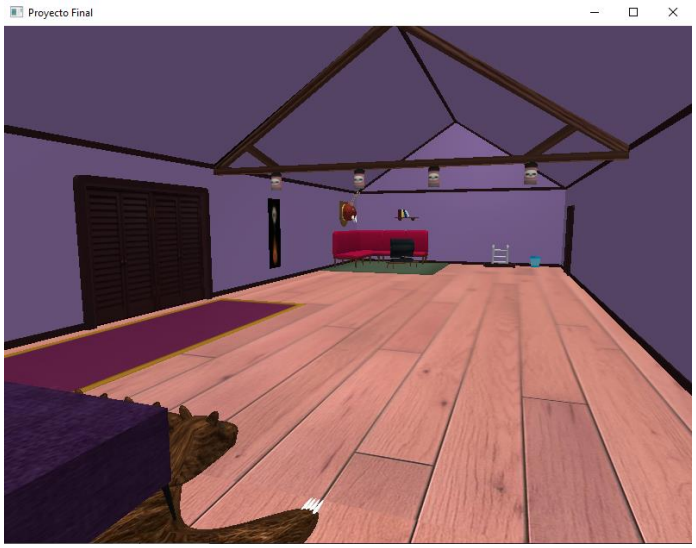
4. Object:

5. Object:

Room 2:

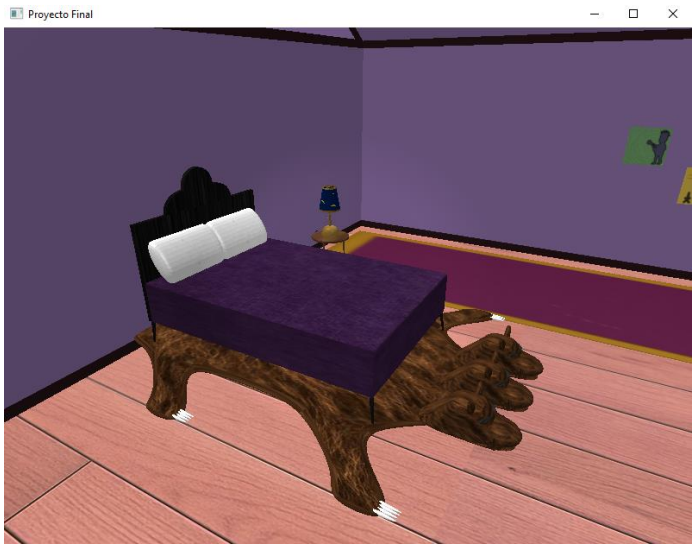

Proyecto Final





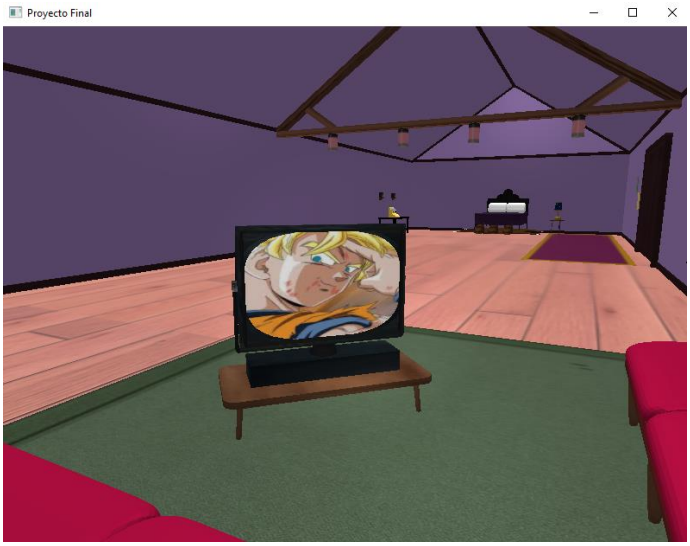
List of items in room 2:

1. Object:



2. Object:

3. Object:



4. Object:

5. Object:

9. Dictionary

9.1 Dictionary of Variables

Name	Type	Description
WIDTH, HEIGHT	const GLuint	Base resolution of the OpenGL window (800×600).
SCREEN_WIDTH, SCREEN_HEIGHT	int	Actual framebuffer resolution of the window.
Camera	Camera	Main camera object (controls view, position, and orientation).
lastX, lastY	GLfloat	Last mouse position used to calculate movement offsets.
keys[1024]	Bool	Array storing the state of all keys (pressed/released).
firstMouse	Bool	Prevents abrupt jumps on the first mouse movement.
lightPos	glm::vec3	Initial position of a point light.
Active	Bool	Enables or disables the animated light.
pointLightPositions[]	glm::vec3	Array containing positions of point lights.
vertices[]	float	Array of vertices for models drawn with OpenGL.
indices[]	GLuint	Index array for drawing models using an EBO.
Light1	glm::vec3	Base color of the animated light that changes over time.
MAX_FRAMES	#define (int)	Maximum number of frames used for animations.
i_max_steps	int	Maximum number of steps in progressive animations.
i_curr_steps	int	Step counter for an animation.
desplazamientoSilla *(chairOffset)*	float	Current position offset of an animated chair.
direccionSilla *(chairDirection)*	int	Movement direction of the chair (1 = +X, −1 = −X, 0 = idle).

Name	Type	Description
WIDTH, HEIGHT	const GLuint	Base resolution of the OpenGL window (800×600).
SCREEN_WIDTH, SCREEN_HEIGHT	int	Actual framebuffer resolution of the window.
Camera	Camera	Main camera object (controls view, position, and orientation).
lastX, lastY	GLfloat	Last mouse position used to calculate movement offsets.
keys[1024]	Bool	Array containing the state of all keys (pressed/released).

firstMouse	Bool	Prevents abrupt jumps during the first mouse movement.
lightPos	glm::vec3	Initial position of a point light.
Active	Bool	Enables or disables the animated light.
pointLightPositions[]	glm::vec3	Array with the positions of point lights.
vertices[]	float	Array of vertices for models rendered with OpenGL.
indices[]	GLuint	Array of drawing indices for models using an EBO.
Light1	glm::vec3	Base color of the animated light that changes over time.
MAX_FRAMES	#define (int)	Maximum number of frames used for animations.
i_max_steps	int	Maximum number of steps in progressive animations.
i_curr_steps	int	Step counter for an animation.
desplazamientoSilla *(chairOffset)*	float	Current displacement position of the animated chair.
direccionSilla *(chairDirection)*	int	Chair movement direction (1 = +X, −1 = −X, 0 = idle).

Name	Type	Description
deltaTime	GLfloat	Time between the current frame and the previous one.
lastFrame	GLfloat	Timestamp of the previous frame.
penduloAngulo *(pendulumAngle)*	float	Current angle of the pendulum.
penduloVelocidad *(pendulumSpeed)*	float	Oscillation speed of the pendulum.
penduloAmplitud *(pendulumAmplitude)*	float	Maximum amplitude in degrees of the pendulum.
bodyY	float	Vertical position of the bat's body.
wingRot	float	Current wing rotation.
headRot	float	Current head rotation.
up	bool	Flag for vertical motion (going up/down).
wingUp	bool	Flag for wing animation (opening/closing).
bodyX	float	X-axis position for the bat's elliptical movement.
bodyZ	float	Z-axis position for the bat's elliptical movement.
bodyRotY	float	Bat rotation according to flying direction.
batState	int	State machine (0 = idle, 1 = flying).
timeElapsed	float	Accumulated time used for bat animation.
ellipseA	float	Major radius of the flight ellipse (X-axis).
ellipseB	float	Minor radius of the flight ellipse (Z-axis).
anguloPuerta *(doorAngle)*	float	Current opening/closing angle of the door.
velocidadPuerta *(doorSpeed)*	float	Opening speed of the door (degrees/second).

abrirPuerta *(openDoor)*	bool	Flag to begin the opening animation.
cerrarPuerta *(closeDoor)*	bool	Flag to begin the closing animation.
tiempoTotal *(totalTime)*	float	Global time for smooth animations (oscillations).
escalaModelo1 *(modelScale1)*	float	Scale factor for model 1 (example: table).

These are the variables used in the main function:

Name	Type	Description
Window	GLFWwindow*	Window control handler.
Projection	glm::mat4	Projection matrix.
VBO, VAO, EBO	GLuint	Buffers for the main 3D model.
skyboxVBO, skyboxVAO	GLuint	Buffers used for rendering the skybox.
skyboxVertices[]	GLfloat[]	Vertex array for the skybox geometry.
faces	std::vector<char*>	File paths for the skybox textures.
cubemapTexture	GLuint	Cubemap texture ID for the skybox.
lightingShader, lampShader, skyboxShader	Shader	Shader programs used for lighting, lamp, and skybox rendering.
Model …	Model	3D world objects loaded through ASSIMP.
texture1..14	GLuint	Loaded texture IDs.
Velocidad *(speed)*	float	Animation speed for the chair movement.
model, modelTemp, modelTemp1	glm::mat4	Transformation matrices.
lightColor	glm::vec3	Dynamic light color.
modelLoc, viewLoc, projLoc	GLint	Uniform locations for transformation matrices.
currentFrame	float	Current time value.
deltaTime	float	Time elapsed between frames.

9.2 Dictionary of Functions

Function Name	Type	Description
KeyCallback()	void	Handles all keyboard input. Detects key presses/releases and triggers functions such as camera movement, light control, chair animation, and door opening/closing.
MouseCallback()	void	Processes mouse movement to control first-person camera orientation. Updates X and Y offsets.
DoMovement()	void	Applies continuous camera movement using W, A, S, D or arrow keys. Also allows camera rotation using Q and E.
Animation()	void	Controls automatic animations such as the character's elliptical motion, body vibration, wing rotation, and head oscillation.

Dibujar()	glm::mat4	Draws a basic cube applying translation and scale. Useful for objects without rotation. Returns the original (pre-modified) model matrix.
DibujarR()	glm::mat4	Draws a cube with rotation, scale, and translation. Ideal for moving parts such as arms, propellers, or mechanical components.
DibujarC()	glm::mat4	Draws a custom box with rotation, scale, translation, and configurable vertex count (depending on the selected face).
DibujarMu()	glm::mat4	Draws a full 3D model applying rotation, scale, and translation. Does not draw vertices directly; only sends the transformation matrix to the shader.
DibujarM()	glm::mat4	A simplified version of **DibujarMu()**, also applying translation, rotation, and scale, returning the transformed model matrix.
DibujarMu2()	glm::mat4	Alternative transformation pipeline for models, using a different order of operations. Returns the final model matrix instead of the original.
loadTexture()	GLuint	Loads a texture from a file using **stb_image**. Configures filtering, wrapping, blending, and generates mipmaps. Returns the texture ID.

10. Code Documentation

10.1 Libraries

These are the libraries that help us recreate the 3D scene of our project, whether for texturing, modeling, or navigating the screen:

```
#include <iostream>
#include <cmath>
// GLEW
#include <GL/glew.h>
// GLFW
#include <GLFW/glfw3.h>
// Other Libs
#include "stb_image.h"
// GLM Mathematics
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
//Load Models
#include "SOIL2/SOIL2.h"
// Other includes
#include "Shader.h"
#include "Camera.h"
#include "Model.h"
#include "Texture.h"
```

These are the same libraries we worked with throughout the course. The only additional one I included was **Texture.h**, which I use to implement the Skybox.

10.2 Global Variables

Let's begin by defining the most important global variables that assist in modeling.

I consider these variables very important because they allow me to scale the models created with OpenGL code. This makes furnishing the house easier and helps me define the proper size of each object:

```
//escalas de modelos de OpenGL
float escalaModelo1 = 0.5f;  // aumentar o disminuir la mesa
float escalaModelo2 = 0.5f;
float escalaModelo3 = 0.25f;
float escalaModelo4 = 0.25f;
float escalaModelo5 = 0.5f;
float escalaModelo6 = 0.25f;
float escalaModelo7 = 0.25f;
float escalaModelo8 = 0.3f;
```

I also have animation variables, for example those used for the bat:

```
// Variables de animación, murciélago
float bodyY = 0.0f;
float wingRot = 0.0f;
float headRot = 0.0f;
bool up = true;
bool wingUp = true;
float bodyX = 0.0f;
float bodyZ = 0.0f;
float bodyRotY = 0.0f;
// Máquina de estados
int batState = 0;  // 0: idle, 1: volando
float timeElapsed = 0.0f;
float ellipseA = 2.5f;   // Radio en eje X
float ellipseB = 1.5f;   // Radio en eje Z
```

10.3 Vertex Arrays

In the code, we have two vertex arrays: one for objects modeled using geometric transformations:

```
//Modelado de Objetos
float vertices[] = {
         -0.5f, -0.5f, -0.5f,  1.0f,  1.0f,  1.0f,  0.0f,0.0f,//frente
```

And another one for the Skybox:

```
//Skybox
GLfloat skyboxVertices[] = {
      // Positions
      -1.0f,  1.0f, -1.0f,
```

It is important to mention here that the EBO must be configured or assigned to the indices of the modeled vertices, because I initially had issues due to incorrect configuration:

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER,  sizeof(indices),  indices,
GL_STATIC_DRAW);
```

10.4 Modeling Functions

I decided to implement functions to handle geometric transformations in order to reduce the number of repeated lines inside the main() function. These functions make it easier to apply transformations and implement hierarchical modeling in a clear and structured way for anyone reading or modifying the project.

10.4.1 Functions for OpenGL Code-Based Models

This is one of the first functions I programmed. At first, it only handled scaling and translation. As I modeled more objects, I needed to create additional functions with parameters for rotation and triangle count.

```
glm::mat4 Dibujar(glm::mat4 model, glm::vec3 escala, glm::vec3 traslado,
GLint uniformModel) {
	glm::mat4 modelTemp = model; // copia la transformación actual
	modelTemp= model = glm::translate(modelTemp, traslado);
	model = glm::scale(model, escala);
	glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
	glDrawArrays(GL_TRIANGLES, 0, 36);

	return modelTemp; // devolvemos la transformación
}
```

10.4.2 Functions for OBJ Models

This works similarly to the function used for OpenGL-coded models, except this one handles scaling, translation, and rotation for models exported from Blender.

```
glm::mat4 DibujarM(glm::mat4 model, glm::vec3 traslado, glm::vec3 escala,
glm::vec3 rotacion, float angulo, GLint modelLoc) {
	glm::mat4 modelTemp = model; // copia la transformación actual
	modelTemp = model = glm::translate(modelTemp, traslado);
	model = glm::rotate(model, glm::radians(angulo), rotacion);
	model = glm::scale(model, escala);

	glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

	return modelTemp; // devolvemos la transformación
```

10.4.3 Texture Loading Function

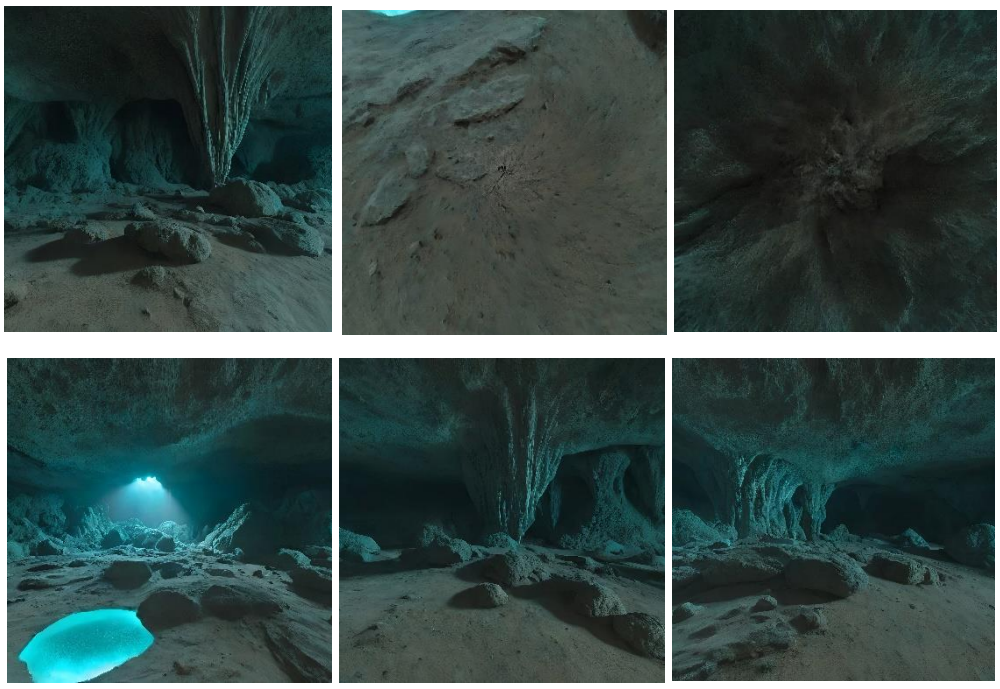
This function is, in my opinion, the most important one because it allows me to generate texture IDs and later assign them to my OpenGL code-based models. (Truthfully, this was one of the most difficult parts because I initially had no idea how to implement more than one texture.)

```
GLuint loadTexture(const char* path) {
	glEnable(GL_BLEND);
	glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
	...
```

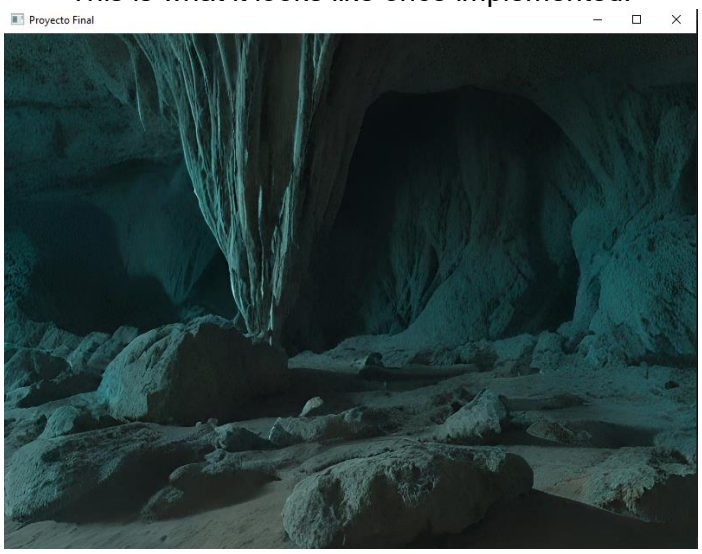
10.5 Skybox

Designing the Skybox for my project was not an easy task. It required obtaining several files from a YouTube channel, where you must fill out a form in order to receive the necessary resources for its implementation.

By following the steps explained in the video, I was able to implement my Skybox. To divide the texture into the six faces, I used a link published in a Reddit forum.



This is what it looks like once implemented:



Explaining the most important part: designing a custom VAO for the skybox. What made this especially difficult was that the skybox was loading correctly, but the textures I created in code were not appearing. I eventually realized the problem was that I had declared the EBO twice—one for each vertex array. However, using **only one EBO**, specifically the one for the models defined in code, is what solved the issue.

In summary, the errors I encountered were the following:

- If you use the same VAO as your models, the skybox renders incorrectly or does not appear.
- If you enable extra attributes (normals/UVs), the textures become corrupted.
- If you don't remove the translation component from the view matrix, the skybox appears to move with you.
- If you load the cubemap using the wrong VAO, you lose textures either from the models or from the skybox.

10.6 Models Created in Code (OpenGL)

The functions previously described are used here as well. It mainly consists of assigning parameters for position, orientation, and size.
The models I created in code were:

1. A table with two chairs
2. A refrigerator
3. A cabinet with a microwave
4. A bookshelf
5. A framed portrait
6. A cardboard box with VHS tapes
7. A clock with a pendulum

10.7 Models Created in Blender

Likewise, the functions implemented for positioning, rotating, and scaling exported Blender objects were used.
The objects I created were:

1. A tombstone
2. A mushroom
3. Two shelves with candles
4. A trash can
5. A bat
6. A table with a type of radio transmitter
7. A cave-like stone object that falls to the ground

10.8 Simple Animations

10.8.1 Animated Head

A simple rotation transformation is applied to simulate the expected movement.

10.8.2 Moving Chairs

They simulate someone about to sit down and pulling the chairs backward. This animation only uses a translation.

10.8.3 Opening Door

The last simple animation is a cabinet door that opens and can close again.

10.9 Complex Animations

10.9.1 Flying Bats

The bat animation in the project is based on trigonometric functions to simulate three main aspects:

1. Wing flapping
2. Body movement (up and down)
3. Flight trajectory (rotation and displacement)

These functions were chosen because they generate smooth, periodic, and natural movements, similar to those seen in real animals.

11. Cost Analysis

Considering the development of the project over a period of three months, the following cost analysis was obtained.

Labor

- **Developer:** The project was carried out by a single developer, responsible for modeling, texturing, programming, and documentation.
  A net monthly compensation of **$12,500.00 MXN** was considered.

Tools and Software

- **Licenses:** For the development of the project, it was necessary to use a 3D design software license for **2 months**, with a monthly cost of **$2,800 MXN**.

Indirect Expenses

- **Electricity:** For the execution of the project, the monthly cost is **$350 MXN**.
- **Internet:** The monthly service cost is **$450 MXN**.

Asset Depreciation

Computer equipment:

- **Original cost:** $28,000 MXN
- **Annual tax depreciation rate:** 30%

$$\text{Depreciación mensual} = \frac{(28,000)(0.30)}{12} = \$700 \text{ MXN}$$

Office equipment:

- **Original cost:** $3,200 MXN
- **Annual tax depreciation rate:** 10%

$$\text{Depreciación mensual} = \frac{(3,200)(0.10)}{12} = \$26.66 \text{ MXN}$$

CATEGORY	COST (MXN)
LABOR (3 MONTHS)	$37,500.00
SOFTWARE LICENSE (2 MONTHS)	$5,600.00
ELECTRICITY AND INTERNET (3 MONTHS)	$2,400.00
COMPUTER EQUIPMENT DEPRECIATION (3 MONTHS)	$2,100.00
OFFICE EQUIPMENT DEPRECIATION (3 MONTHS)	$79.98
ESTIMATED TOTAL WITHOUT PROFIT	$47,679.98
CONTINGENCY (10%)	$4,767.99
SUBTOTAL	$52,447.97
PROFIT (12%)	$6,293.75
SUBTOTAL WITH PROFIT	$58,741.72
VAT (16%)	$9,398.67
TOTAL WITH VAT	$68,140.39

12. State of the Art of the Project

Virtual tours have become an essential tool in various fields such as entertainment, architecture, education, and real estate. Their popularity has increased due to advancements in graphics technologies, 3D engines, virtual reality devices, and optimized techniques for representing digital spaces in increasingly immersive and realistic ways. Currently, virtual tours allow users to explore three-dimensional environments without physical limitations, making it possible to visualize spaces that do not exist, are difficult to access, or are entirely fictional. In the entertainment and videogame industries, virtual tours are used to create interactive worlds that provide dynamic and engaging experiences. Iconic animated series, movies, and franchises are often recreated in digital environments by fans or professional artists, allowing users to explore fictional locations with a high degree of visual and aesthetic freedom. In the architectural field, virtual tours have become a standard tool for presenting projects before construction, improving decision-making and facilitating communication between clients and designers. Likewise, in education, they are used to recreate historical or scientific environments that enrich teaching-learning processes.

In this context, the development of a virtual tour of Marceline's house—character from the series *Adventure Time*—aligns with current trends in digital recreation of fictional spaces. This project includes the modeling of the main structure of the house along with ten additional objects—including a bed, lamps, and decorative elements—based on original reference images to achieve accurate colors and proportions. Additionally, four animations were integrated to add dynamism to the scene, following modern practices in interactive design for 3D environments.

The use of modeling, texturing, and animation tools, together with lighting and rendering techniques, ensures that this virtual tour maintains aesthetic coherence with the original work while taking advantage of current advances in three-dimensional visualization. This type of project demonstrates how virtual tours not only replicate real spaces but also serve as a creative medium for interpreting imaginary worlds with a high degree of detail and immersion.

13. Conclusions

Through the development of this project, the modeling and animation stages turned out to be the most enjoyable and enriching parts. Implementing transformations, trajectories, and complex movements allowed me to apply fundamental concepts of computer graphics and see immediate visual results. Additionally, creating the skybox and the overall environment helped me understand how small elements can significantly enhance the perception of a scene.

This project also showed me the impact that computer graphics have on the user experience. Details such as the shape of a model, the lighting, or the quality of the texturing can generate acceptance or rejection in whoever views the scene. For this reason, I made an effort to take care of the façade details and to create a coherent visual environment, aiming to convey a strong first impression.

Overall, this work strengthened my understanding that computer graphics is not only about drawing objects on a screen, but about making design decisions that directly influence how people perceive and interpret a space. Every technical or aesthetic choice has an impact, and mastering these processes makes it possible to create more solid, appealing, and meaningful experiences.

14. References

- Crane, L. (n.d.). *Panorama to Cubemap* [Web tool]. Retrieved from: https://jaxry.github.io/panorama-to-cubemap/
- *Implementación de un Skybox en OpenGL: Cielo y Ambiente Realista para Escenas 3D* [Video]. (2025, …). YouTube. Retrieved from: https://youtu.be/VDfRE9VjeAA?si=pPld2EmGDU3kRFeY